HTML Complete Notes — Part 3: HTML Forms

Table of Contents

42. HTML Forms
43. HTML Form Attributes
44. HTML Form Elements
45. HTML Input Types
46. HTML Input Attributes
47. Input Form Attributes
48. Complete HTML Form Project
49. Practice Questions with Solutions
50. Important Viva Questions

---

42. HTML Forms

42.1 What is an HTML form?

An HTML form is a section of a web page that collects information from users.

Forms allow users to enter data, select options, upload files, and submit information.

Examples of forms include:

- Login forms
- Registration forms
- Contact forms
- Feedback forms
- Search forms
- Online application forms
- Payment forms

The `<form>` element is used to create a form.

42.2 Basic syntax

```
<form action="/submit" method="post">
    <!-- Form controls go here -->
</form>
```

Explanation

- `<form>` defines the form.
- `action` specifies the URL that will process the submitted data.
- `method` specifies how the browser sends the form data.
- The content inside `<form>` contains controls such as text fields, radio buttons, and submit buttons.

The URL in this example is illustrative. A working submission requires a server endpoint that accepts and processes the request.

42.3 Your first HTML form

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>My First Form</title>
</head>
<body>

    <h1>Student Registration</h1>

    <form action="/register" method="post">

        <label for="fullname">Full Name:</label>
        <input type="text" id="fullname" name="fullname">

        <br><br>

        <label for="email">Email:</label>
        <input type="email" id="email" name="email">

        <br><br>

        <button type="submit">Register</button>

    </form>

</body>
</html>
```

Understanding the form step by step

1. The user opens the registration page.
2. The user enters a name and email address.
3. The browser performs applicable client-side validation.
4. When the user clicks Register, the browser prepares the successful form controls for submission.
5. The browser sends the data to `/register` using the POST method.
6. The server must receive and process the request.

Important: HTML creates the form interface, but it does not automatically create a database record. Saving data requires a backend or another appropriate service.

42.4 What is a label?

A `<label>` describes the purpose of a form control.

```
<label for="username">Username:</label>
<input type="text" id="username" name="username">
```

The `for` value must match the control's `id`.

Benefits:

- Makes the form easier to understand.
- Helps screen-reader users identify controls.
- Clicking the label can focus or activate its associated control.

42.5 What is the purpose of the `name` attribute?

The `name` attribute identifies the field when the form data is submitted.

```
<input type="text" name="username">
```

If the user enters `Prajwal`, the submitted data can contain:

```
username=Prajwal
```

A control generally needs a `name` to contribute a field to normal form submission.

The `id` and `name` attributes are different:

Attribute	Purpose
`id`	Identifies the element in the document.
`name`	Identifies the submitted field.

---

43. HTML Form Attributes

Form attributes control where and how the browser submits form data.

43.1 The `action` attribute

The `action` attribute specifies the URL that receives the submitted form data.

```
<form action="/register" method="post">
    <input type="text" name="username">
    <button type="submit">Register</button>
</form>
```

When submitted, the browser sends the form data to `/register`.

If `action` is omitted, the form generally submits to the current document URL.

43.2 The `method` attribute

The `method` attribute specifies the HTTP method used to submit the form.

The two most commonly used values are `get` and `post`.

GET method

```
<form action="/search" method="get">
    <label for="query">Search:</label>
    <input type="search" id="query" name="q">

    <button type="submit">Search</button>
</form>
```

If the user searches for HTML, the URL may look like:

```
/search?q=HTML
```

Characteristics:

- Form data is generally included in the URL query string.
- Useful for searches and other read-only requests.
- URLs can be bookmarked and shared.
- Not suitable for secrets because query values can appear in browser history, logs, and URLs.

POST method

```
<form action="/register" method="post">
    <label for="username">Username:</label>
    <input type="text" id="username" name="username">

    <button type="submit">Register</button>
</form>
```

Characteristics:

- Form data is sent in the request body for a typical POST form submission.
- Commonly used to create or update server-side data.
- Data is not ordinarily displayed in the URL query string.
- POST does not automatically make data secure. HTTPS and appropriate server-side security are still necessary.

GET vs. POST

GET	POST
Commonly used for searching and retrieving information.	Commonly used for submitting data that may change server state.
Data is usually included in the URL query string.	Data is usually included in the request body.
Requests can often be bookmarked and shared.	Request bodies are not normally preserved as part of a bookmark.
Not suitable for confidential information.	Does not itself provide confidentiality.
Usually used for read-only operations.	Often used for operations that create or modify data.

43.3 The `enctype` attribute

`enctype` specifies how the form data is encoded when using POST.

Default encoding

```
<form action="/submit" method="post">
    <input type="text" name="username">
    <button type="submit">Submit</button>
</form>
```

The default is:

`application/x-www-form-urlencoded`

Multipart encoding

For file uploads, use:

```
<form action="/upload"
      method="post"
      enctype="multipart/form-data">

    <label for="document">Choose a document:</label>
    <input type="file" id="document" name="document">

    <button type="submit">Upload</button>
</form>
```

The server must support file uploads and process the multipart request.

Plain-text encoding

`text/plain` is another possible encoding, but it is rarely appropriate for production forms.

43.4 The `target` attribute

The `target` attribute specifies where the response should be displayed.

```
<form action="/search" method="get" target="_blank">
    <input type="search" name="q">
    <button type="submit">Search</button>
</form>
```

Common values include:

- `_self`: Current browsing context.
- `_blank`: A new browsing context.
- `_parent`: Parent browsing context.
- `_top`: Top-level browsing context.

43.5 The `autocomplete` attribute

This attribute controls whether the browser may offer autocomplete suggestions for form fields.

```
<form autocomplete="on">
    <label for="name">Name:</label>
    <input type="text" id="name" name="name">
</form>
```

You can also use it on individual controls:

```
<input type="email"
       name="email"
       autocomplete="email">
```

Common values include `on`, `off`, `name`, `email`, `username`, and `current-password`.

43.6 The `novalidate` attribute

This attribute disables the browser's built-in constraint validation when submitting the form.

```
<form action="/submit" method="post" novalidate>
    <input type="email" name="email" required>
    <button type="submit">Submit</button>
</form>
```

Even though the field is marked `required`, the browser will not block submission because of that constraint when the form is submitted normally.

The server should still validate all received data.

43.7 The `accept-charset` attribute

This attribute specifies the character encoding used for form submission.

```
<form action="/submit" method="post" accept-charset="UTF-8">
    <input type="text" name="name">
    <button type="submit">Submit</button>
</form>
```

UTF-8 is the standard practical choice for modern web forms.

43.8 Complete example using form attributes

```
<form action="/register"
      method="post"
      autocomplete="on"
      accept-charset="UTF-8">

    <label for="fullname">Full Name:</label>
    <input type="text"
           id="fullname"
           name="fullname"
           autocomplete="name"
           required>

    <button type="submit">Register</button>
</form>
```

---

44. HTML Form Elements

Form elements are the controls and supporting elements used to collect user input.

44.1 The `<input>` element

The `<input>` element creates many types of controls.

```
<input type="text" name="username">
```

The `type` attribute determines the kind of input.

44.2 The `<label>` element

```
<label for="age">Age:</label>
<input type="number" id="age" name="age">
```

The label explains what the control is for.

44.3 The `<textarea>` element

A `<textarea>` allows users to enter multiline text.

```
<label for="message">Message:</label>

<textarea id="message"
          name="message"
          rows="5"
          cols="30"></textarea>
```

Unlike an ordinary text input, a textarea supports multiple lines.

44.4 The `<select>` element

A `<select>` creates a dropdown menu.

```
<label for="city">Choose a city:</label>

<select id="city" name="city">
    <option value="mumbai">Mumbai</option>
    <option value="pune">Pune</option>
    <option value="nashik">Nashik</option>
</select>
```

The submitted value comes from the selected option's `value` attribute.

44.5 The `<option>` element

An `<option>` represents one selectable item in a dropdown or list.

```
<select name="course">
    <option value="">Select a course</option>
    <option value="computer">Computer Engineering</option>
    <option value="mechanical">Mechanical Engineering</option>
</select>
```

44.6 The `<optgroup>` element

Use `<optgroup>` to group related dropdown options.

```
<label for="course">Course:</label>

<select id="course" name="course">
    <optgroup label="Engineering">
        <option value="computer">Computer Engineering</option>
        <option value="mechanical">Mechanical Engineering</option>
    </optgroup>

    <optgroup label="Science">
        <option value="physics">Physics</option>
        <option value="chemistry">Chemistry</option>
    </optgroup>
</select>
```

44.7 The `<button>` element

```
<button type="submit">Submit Form</button>
```

A button can submit a form, reset controls, or perform a custom action.

44.8 The `<fieldset>` element

A `<fieldset>` groups related form controls.

```
<fieldset>
    <legend>Personal Information</legend>

    <label for="fname">First Name:</label>
    <input type="text" id="fname" name="fname">

    <label for="lname">Last Name:</label>
    <input type="text" id="lname" name="lname">
</fieldset>
```

44.9 The `<legend>` element

The `<legend>` provides a caption for a `<fieldset>`.

```
<fieldset>
    <legend>Account Details</legend>
    <input type="text" name="username">
</fieldset>
```

44.10 The `<datalist>` element

A `<datalist>` provides suggestions for an input. Users can generally enter other values too.

```
<label for="browser">Choose a browser:</label>

<input list="browsers" id="browser" name="browser">

<datalist id="browsers">
    <option value="Chrome">
    <option value="Firefox">
    <option value="Edge">
    <option value="Safari">
</datalist>
```

Unlike a `<select>`, a datalist does not restrict the user to its listed suggestions.

44.11 The `<output>` element

The `<output>` element represents the result of a calculation or user action.

```
<form oninput="result.value = Number(a.value) + Number(b.value)">
    <input type="number" id="a" name="a" value="0">
    +
    <input type="number" id="b" name="b" value="0">
    =
    <output name="result" for="a b">0</output>
</form>
```

Try changing the numbers. The result updates automatically.

Summary of form elements

Element	Purpose
`<form>`	Contains the form controls.
`<input>`	Creates different input controls.
`<label>`	Describes a control.
`<textarea>`	Collects multiline text.
`<select>`	Creates a dropdown or selection list.
`<option>`	Defines a selectable option.

Element	Purpose
`<optgroup>`	Groups related options.
`<button>`	Creates a button.
`<fieldset>`	Groups related controls.
`<legend>`	Labels a fieldset.
`<datalist>`	Provides input suggestions.
`<output>`	Represents a result.

---

45. HTML Input Types

The `type` attribute determines how an `<input>` behaves and appears.

45.1 Text input

```
<label for="username">Username:</label>
<input type="text" id="username" name="username">
```

Used for a single line of ordinary text.

45.2 Password input

```
<label for="password">Password:</label>
<input type="password" id="password" name="password">
```

Characters are visually obscured, but this does not encrypt the data or provide authentication by itself.

45.3 Email input

```
<label for="email">Email:</label>
<input type="email" id="email" name="email" required>
```

Provides email-oriented input behavior and basic browser validation.

It does not prove that an email address exists.

45.4 Number input

```
<label for="age">Age:</label>
<input type="number"
       id="age"
       name="age"
       min="1"
       max="120">
```

Useful for numeric values.

45.5 Radio buttons

Radio buttons let users select one option from a group.

```
<p>Gender:</p>

<label>
    <input type="radio" name="gender" value="male">
    Male
</label>

<label>
    <input type="radio" name="gender" value="female">
    Female
</label>

<label>
    <input type="radio" name="gender" value="other">
    Other
</label>
```

Radio buttons belong to the same group when they share the same `name`.

Only the selected radio button in a group is normally submitted.

45.6 Checkboxes

Checkboxes let users independently select multiple options.

```
<p>Select your skills:</p>

<label>
    <input type="checkbox" name="skills" value="html">
    HTML
</label>

<label>
    <input type="checkbox" name="skills" value="css">
    CSS
</label>

<label>
    <input type="checkbox" name="skills" value="javascript">
    JavaScript
</label>
```

If several boxes are checked, multiple values can be submitted under the same name.

45.7 Date input

```
<label for="dob">Date of Birth:</label>
<input type="date" id="dob" name="dob">
```

The browser typically displays a date picker.

45.8 Time input

```
<label for="meeting">Meeting Time:</label>
<input type="time" id="meeting" name="meeting">
```

45.9 Datetime-local input

```
<label for="appointment">Appointment:</label>
<input type="datetime-local"
       id="appointment"
       name="appointment">
```

Collects a local date and time without specifying a timezone.

45.10 Month input

```
<label for="month">Choose a month:</label>
<input type="month" id="month" name="month">
```

45.11 Week input

```
<label for="week">Choose a week:</label>
<input type="week" id="week" name="week">
```

45.12 Color input

```
<label for="color">Choose a color:</label>
<input type="color" id="color" name="color" value="#0000ff">
```

45.13 Range input

```
<label for="volume">Volume:</label>
<input type="range"
       id="volume"
       name="volume"
       min="0"
       max="100"
       value="50">
```

Useful for selecting a value within a range.

45.14 File input

```
<label for="resume">Upload Resume:</label>
<input type="file" id="resume" name="resume">
```

To allow multiple files:

```
<input type="file" name="documents" multiple>
```

To suggest PDF files:

```
<input type="file"
       name="resume"
       accept=".pdf,application/pdf">
```

The `accept` attribute is a hint to the file picker, not a security guarantee. The server must validate uploaded files.

45.15 Hidden input

```
<input type="hidden" name="form_version" value="registration-v1">
```

Hidden inputs are not displayed like normal controls, but their values may be submitted.

Important: Hidden inputs are not secret. Users can inspect and modify them.

45.16 Search input

```
<label for="search">Search:</label>
<input type="search" id="search" name="q">
```

45.17 Telephone input

```
<label for="phone">Phone Number:</label>
<input type="tel" id="phone" name="phone">
```

This can provide a telephone-friendly keyboard on mobile devices. It does not enforce a universal phone number format automatically.

45.18 URL input

```
<label for="website">Website:</label>
<input type="url" id="website" name="website">
```

Provides basic browser validation for URL-like values.

45.19 Submit input

```
<input type="submit" value="Submit Form">
```

Submits the form when activated.

45.20 Reset input

```
<input type="reset" value="Reset Form">
```

Resets form controls to their initial values. Use reset buttons carefully because users may accidentally discard their input.

45.21 Button input

```
<input type="button" value="Click Me">
```

This creates a general-purpose button. It needs JavaScript or another relevant behavior to perform a custom action.

45.22 Image input

```
<input type="image"
       src="submit.png"
       alt="Submit form"
       width="100">
```

An image input acts as a graphical submit button.

Summary of input types

Type	Purpose
`text`	Single-line text.
`password`	Password-style input.
`email`	Email address.
`number`	Numeric value.
`radio`	One selection from a group.
`checkbox`	Independent selections.
`date`	Calendar date.
`time`	Time of day.
`datetime-local`	Local date and time.
`month`	Month and year.
`week`	Week and year.
`color`	Color selection.

Type	Purpose
`range`	Value within a range.
`file`	File selection.
`hidden`	Hidden submitted value.
`search`	Search text.
`tel`	Telephone-oriented input.
`url`	URL-oriented input.
`submit`	Submit button.
`reset`	Reset button.
`button`	General-purpose button.
`image`	Graphical submit button.

---

46. HTML Input Attributes

Input attributes configure the behavior, validation, and initial values of form controls.

46.1 The `value` attribute

Specifies an initial value for many input types.

```
<input type="text" name="username" value="Prajwal">
```

The field initially displays `Prajwal`.

For checkboxes and radio buttons, `value` specifies the value submitted when the control is selected.

```
<input type="radio" name="course" value="html">
```

46.2 The `placeholder` attribute

Displays a short hint when the field is empty.

```
<input type="text"
       name="username"
       placeholder="Enter your username">
```

A placeholder is not a substitute for a visible label.

46.3 The `required` attribute

Makes a control required for normal browser form validation.

```
<input type="email" name="email" required>
```

The browser can prevent submission when the required field is empty.

The server must still validate submitted data.

46.4 The `readonly` attribute

Makes a supported control non-editable by the user.

```
<input type="text"
       name="student_id"
       value="STU101"
       readonly>
```

A read-only control can still be submitted if it has a name.

46.5 The `disabled` attribute

Disables a control.

```
<input type="text" name="username" disabled>
```

A disabled control is generally not focusable, does not participate in ordinary constraint validation, and is not included in form submission.

Readonly vs. disabled

Readonly	Disabled
Cannot normally be edited by the user.	Cannot normally be interacted with.
Can generally receive focus.	Normally cannot receive focus.
Can be submitted if it has a name.	Is not submitted as a successful control.

Readonly	Disabled
Available only on supported control types.	Available on many form controls.

46.6 The `maxlength` attribute

Sets the maximum number of characters for supported text inputs.

```
<input type="text"
       name="username"
       maxlength="20">
```

46.7 The `minlength` attribute

Sets the minimum number of characters for supported text inputs.

```
<input type="text"
       name="username"
       minlength="3"
       maxlength="20">
```

46.8 The `min` and `max` attributes

Set the allowed minimum and maximum for supported numeric and date-related inputs.

```
<input type="number"
       name="age"
       min="18"
       max="60">
```

The browser can reject values outside the specified range during normal constraint validation.

46.9 The `step` attribute

Defines the permitted increments for supported numeric and date/time inputs.

```
<input type="number"
       name="price"
       min="0"
       step="0.01">
```

This allows values such as `10.25` and `10.50`.

For a number input, the default step is generally `1`, unless another rule applies.

46.10 The `pattern` attribute

Defines a regular expression used to validate supported text-like input types.

```
<label for="code">Student Code:</label>

<input type="text"
       id="code"
       name="code"
       pattern="[A-Z]{2}[0-9]{4}"
       title="Enter two uppercase letters followed by four digits"
       required>
```

Example accepted value: `AB1234`.

The `pattern` attribute is not supported as a general validation mechanism for every input type. Use it only with supported text-like types.

46.11 The `checked` attribute

Initially checks a checkbox or radio button.

```
<input type="checkbox" name="subscribe" value="yes" checked>
```

46.12 The `multiple` attribute

Allows multiple values for supported input types, such as email and file.

```
<input type="file" name="files" multiple>
```

Another example:

```
<input type="email" name="emails" multiple>
```

46.13 The `accept` attribute

Suggests which file types should be selectable in a file picker.

```
<input type="file"
       name="photo"
       accept="image/*">
```

This does not guarantee that the uploaded file is safe or genuinely of that type.

46.14 The `autocomplete` attribute

Provides autocomplete hints.

```
<input type="text"
       name="fullname"
       autocomplete="name">

<input type="email"
       name="email"
       autocomplete="email">

<input type="password"
       name="password"
       autocomplete="new-password">
```

46.15 The `autofocus` attribute

Requests that a control receive focus when the page loads.

```
<input type="text" name="username" autofocus>
```

Use it sparingly, as automatic focus can be disruptive for some users.

46.16 The `size` attribute

Specifies the visible width of supported text-like input fields in approximate character units.

```
<input type="text" name="username" size="30">
```

CSS is generally preferable for controlling layout.

46.17 The `list` attribute

Connects an input to a `<datalist>`.

```
<label for="browser">Browser:</label>

<input type="text" id="browser" name="browser" list="browser-options">

<datalist id="browser-options">
    <option value="Chrome">
    <option value="Firefox">
    <option value="Edge">
</datalist>
```

46.18 The `inputmode` attribute

Suggests an appropriate virtual keyboard on devices that support it.

```
<input type="text" name="pin" inputmode="numeric">
```

This can suggest a numeric keyboard, but does not validate the input by itself.

46.19 The `enterkeyhint` attribute

Hints at the label or purpose of the Enter key on virtual keyboards.

```
<input type="text"
       name="search"
       enterkeyhint="search">
```

Input attributes summary

Attribute	Purpose
`value`	Initial or submitted value, depending on input type.
`placeholder`	Displays a short hint.
`required`	Requires a value for normal validation.
`readonly`	Prevents normal user editing.
`disabled`	Disables the control.
`maxlength`	Sets maximum text length.
`minlength`	Sets minimum text length.
`min`	Sets a minimum allowed value.

Attribute	Purpose
`max`	Sets a maximum allowed value.
`step`	Sets allowed increments.
`pattern`	Defines a validation pattern for supported types.
`checked`	Initially checks a checkbox or radio.
`multiple`	Allows multiple values for supported types.
`accept`	Suggests file types.
`autocomplete`	Provides autocomplete instructions.
`autofocus`	Requests initial focus.
`size`	Sets approximate visible input width.
`list`	Connects to a datalist.
`inputmode`	Suggests a virtual keyboard.
`enterkeyhint`	Hints at the virtual Enter key action.

---

47. Input Form Attributes

Input form attributes are attributes that allow an individual form control to specify submission behavior or associate with a form.

These are different from the general input attributes covered in the previous section.

47.1 The `form` attribute

The `form` attribute associates a form control with a form using the form's ID.

The control does not have to be physically inside the `<form>` element.

```
<!DOCTYPE html>
<html>
<head>
    <title>Form Attribute</title>
</head>
<body>

    <form id="student-form" action="/submit" method="post">
        <label for="name">Name:</label>
        <input type="text" id="name" name="name">
    </form>

    <label for="email">Email:</label>
    <input type="email"
           id="email"
           name="email"
           form="student-form">

    <button type="submit" form="student-form">
        Submit
    </button>

</body>
</html>
```

The email field and submit button are associated with `student-form`.

This is useful when a page layout requires controls to appear outside the form's immediate markup.

47.2 The `formaction` attribute

Overrides the form's `action` URL for a particular submit button or image submit control.

```
<form action="/save-draft" method="post">
    <input type="text" name="title">

    <button type="submit">Save Draft</button>

    <button type="submit"
            formaction="/publish">
        Publish
    </button>
</form>
```

The first button uses `/save-draft`. The second uses `/publish`.

The server must implement both endpoints.

47.3 The `formenctype` attribute

Overrides the form's encoding type for a particular submit button.

```
<form action="/submit"
      method="post"
      enctype="application/x-www-form-urlencoded">

    <input type="text" name="username">

    <input type="file" name="document">

    <button type="submit"
            formenctype="multipart/form-data">
        Upload
    </button>
</form>
```

The button requests multipart encoding for that submission.

47.4 The `formmethod` attribute

Overrides the form's method for a particular submit button.

```
<form action="/search" method="get">
    <input type="search" name="q">

    <button type="submit">Search with GET</button>

    <button type="submit"
            formmethod="post">
        Submit with POST
    </button>
</form>
```

The second button requests POST rather than GET.

47.5 The `formnovalidate` attribute

Allows a particular submit button to bypass the browser's built-in constraint validation.

```
<form action="/register" method="post">
    <label for="email">Email:</label>
    <input type="email" id="email" name="email" required>

    <button type="submit">Register</button>

    <button type="submit" formnovalidate>
        Submit Without Browser Validation
    </button>
</form>
```

The second button bypasses client-side constraint validation for that submission.

The server should still validate the data.

47.6 The `formtarget` attribute

Overrides the form's target for a particular submit button.

```
<form action="/search" method="get">
    <input type="search" name="q">

    <button type="submit">Open Here</button>

    <button type="submit" formtarget="_blank">
        Open Response in New Tab
    </button>
</form>
```

The second button requests that the response be opened in a new browsing context.

Summary of input form attributes

Attribute	Purpose
`form`	Associates a control with a form by ID.
`formaction`	Overrides the submission URL.
`formenctype`	Overrides the form data encoding.
`formmethod`	Overrides the submission method.
`formnovalidate`	Bypasses browser constraint validation for that submission.
`formtarget`	Overrides where the response is displayed.

Important distinction

- `action`, `method`, and `enctype` are normally form attributes.
- `formaction`, `formmethod`, and `formenctype` are submit-control attributes.
- The corresponding `form*` attributes override the general form settings for a particular submission.

---

48. Complete HTML Form Project

Project: Student Registration Form

This project combines form elements, input types, validation attributes, dropdowns, radio buttons, checkboxes, and file uploads.

Complete HTML program

Save this file as `registration.html`.

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">

    <meta name="viewport"
          content="width=device-width, initial-scale=1.0">

    <title>Student Registration Form</title>

    <style>
        * {
            box-sizing: border-box;
        }

        body {
            font-family: Arial, sans-serif;
            background-color: #f2f5f9;
            margin: 0;
            padding: 20px;
        }

        .container {
            max-width: 650px;
            margin: 20px auto;
            padding: 25px;
            background-color: white;
            border-radius: 10px;
        }

        h1 {
            text-align: center;
            color: navy;
        }

        label {
            display: block;
            margin-top: 14px;
            margin-bottom: 6px;
        }

        input:not([type="radio"]):not([type="checkbox"]),
        select,
        textarea {
            width: 100%;
            padding: 10px;
            border: 1px solid #cccccc;
```

```
            border-radius: 5px;
        }

        .choice-label {
            display: inline-block;
            margin-right: 12px;
        }

        fieldset {
            margin-top: 18px;
            border: 1px solid #cccccc;
            border-radius: 5px;
        }

        button {
            margin-top: 20px;
            padding: 12px 18px;
            border: none;
            border-radius: 5px;
            cursor: pointer;
        }

        .submit-btn {
            background-color: navy;
            color: white;
        }

        .reset-btn {
            background-color: #dddddd;
        }
    </style>
</head>

<body>

    <div class="container">

        <h1>Student Registration</h1>

        <form action="/register"
              method="post"
              autocomplete="on">

            <label for="fullname">Full Name:</label>

            <input type="text"
                   id="fullname"
```

```
        name="fullname"
        placeholder="Enter your full name"
        autocomplete="name"
        minlength="3"
        maxlength="60"
        required>

<label for="email">Email Address:</label>

<input type="email"
        id="email"
        name="email"
        placeholder="Enter your email"
        autocomplete="email"
        required>

<label for="phone">Phone Number:</label>

<input type="tel"
        id="phone"
        name="phone"
        placeholder="Enter your phone number"
        autocomplete="tel">

<label for="dob">Date of Birth:</label>

<input type="date"
        id="dob"
        name="dob">

<fieldset>
    <legend>Gender</legend>

    <label class="choice-label">
        <input type="radio"
               name="gender"
               value="male">
        Male
    </label>

    <label class="choice-label">
        <input type="radio"
               name="gender"
               value="female">
        Female
    </label>
```

```
            <label class="choice-label">
                <input type="radio"
                       name="gender"
                       value="other">
                Other
            </label>
        </fieldset>

        <label for="course">Select Course:</label>

        <select id="course" name="course" required>
            <option value="">Choose a course</option>
            <option value="computer">Computer Engineering</option>
            <option value="mechanical">Mechanical Engineering</option>
            <option value="civil">Civil Engineering</option>
        </select>

        <fieldset>
            <legend>Technical Skills</legend>

            <label class="choice-label">
                <input type="checkbox"
                       name="skills"
                       value="html">
                HTML
            </label>

            <label class="choice-label">
                <input type="checkbox"
                       name="skills"
                       value="css">
                CSS
            </label>

            <label class="choice-label">
                <input type="checkbox"
                       name="skills"
                       value="javascript">
                JavaScript
            </label>
        </fieldset>

        <label for="resume">Upload Resume (PDF):</label>

        <input type="file"
               id="resume"
               name="resume"
```

```
                accept=".pdf,application/pdf">

            <label for="address">Address:</label>

            <textarea id="address"
                      name="address"
                      rows="4"
                      placeholder="Enter your address"></textarea>

            <label>
                <input type="checkbox"
                       name="terms"
                       value="accepted"
                       required>
                I agree to the terms and conditions.
            </label>

            <button type="submit" class="submit-btn">
                Register
            </button>

            <button type="reset" class="reset-btn">
                Reset
            </button>

        </form>

    </div>

</body>
</html>
```

What does this project demonstrate?

1. Text, email, telephone, and date inputs.
2. Radio buttons and checkboxes.
3. Dropdown menus.
4. File selection.
5. Multiline text input.
6. Labels and fieldsets.
7. Required fields and basic validation.
8. CSS styling and responsive container sizing.

9. Submit and reset buttons.

Important project note

The example uses `action="/register"` as a placeholder backend endpoint. The form interface will render in a browser, but successful server-side registration requires a backend implementation.

For a real registration system, validate the submitted data on the server, safely process uploaded files, and use HTTPS.

---

49. Practice Questions with Solutions

Question 1: Create a login form

Requirements:

- Username
- Password
- Submit button

Solution

```
<form action="/login" method="post">
    <label for="username">Username:</label>
    <input type="text"
           id="username"
           name="username"
           required>

    <label for="password">Password:</label>
    <input type="password"
           id="password"
           name="password"
           required>

    <button type="submit">Login</button>
</form>
```

Question 2: Create a form with a dropdown

Requirements:

- Name

- City dropdown
- Submit button

Solution

```
<form action="/submit" method="post">
    <label for="name">Name:</label>
    <input type="text" id="name" name="name" required>

    <label for="city">City:</label>
    <select id="city" name="city" required>
        <option value="">Select City</option>
        <option value="mumbai">Mumbai</option>
        <option value="pune">Pune</option>
        <option value="nashik">Nashik</option>
    </select>

    <button type="submit">Submit</button>
</form>
```

Question 3: Create a form with radio buttons and checkboxes

Solution

```
<form action="/submit" method="post">
    <p>Study Mode:</p>

    <label>
        <input type="radio" name="mode" value="online" required>
        Online
    </label>

    <label>
        <input type="radio" name="mode" value="offline">
        Offline
    </label>

    <p>Interests:</p>

    <label>
        <input type="checkbox" name="interest" value="coding">
        Coding
    </label>

    <label>
        <input type="checkbox" name="interest" value="design">
        Design
    </label>

    <button type="submit">Submit</button>
</form>
```

Question 4: Create a field that accepts only numbers between 18 and 60

Solution

```
<label for="age">Age:</label>

<input type="number"
       id="age"
       name="age"
       min="18"
       max="60"
       required>
```

Question 5: Create a text field with a maximum length of 15 characters

Solution

```
<input type="text"
       name="username"
       maxlength="15">
```

Question 6: Create a field for a student code such as AB1234

Solution

```
<label for="student-code">Student Code:</label>

<input type="text"
       id="student-code"
       name="student_code"
       pattern="[A-Z]{2}[0-9]{4}"
       title="Use two uppercase letters followed by four digits"
       required>
```

Question 7: Create a file upload form

Solution

```
<form action="/upload"
      method="post"
      enctype="multipart/form-data">

    <label for="document">Upload PDF:</label>

    <input type="file"
           id="document"
           name="document"
           accept=".pdf,application/pdf"
           required>

    <button type="submit">Upload</button>
</form>
```

The server must support multipart requests and validate the uploaded file.

---

50. Important HTML Forms Viva Questions

1. What is an HTML form?

An HTML form collects information from users through controls such as input fields, checkboxes, radio buttons, and dropdowns.

2. What is the purpose of the `<form>` tag?

It groups form controls and defines how their data is submitted.

3. What is the difference between GET and POST?

GET normally places submitted form data in the URL query string, while POST generally sends it in the request body.

4. What is the purpose of the `action` attribute?

It specifies the URL that receives the submitted form data.

5. What is the purpose of the `method` attribute?

It specifies the HTTP method used for form submission.

6. What is the difference between `id` and `name`?

`id` identifies an element in the document, while `name` identifies the field in submitted form data.

7. What is the purpose of `<label>`?

It provides an accessible description for a form control.

8. What is the difference between radio buttons and checkboxes?

Radio buttons normally allow one selection per group. Checkboxes allow independent selections.

9. What is the purpose of the `required` attribute?

It makes a control required for normal browser constraint validation.

10. What is the difference between `readonly` and `disabled`?

Readonly controls cannot normally be edited but can generally be submitted. Disabled controls cannot normally be interacted with and are not submitted.

11. What is the purpose of the `placeholder` attribute?

It displays a short hint inside an empty control.

12. What is the purpose of the `maxlength` attribute?

It limits the maximum number of characters in supported text inputs.

13. What is the purpose of the `pattern` attribute?

It defines a regular expression used for client-side validation of supported text-like inputs.

14. What is the purpose of `<textarea>`?

It collects multiline text.

15. What is the purpose of `<select>`?

It creates a dropdown or selection list.

16. What is the purpose of `<fieldset>`?

It groups related form controls.

17. What is the purpose of `<legend>`?

It provides a caption for a fieldset.

18. What is the purpose of the `enctype` attribute?

It specifies how submitted form data is encoded, particularly for POST requests.

19. Why use `multipart/form-data`?

It is commonly used when submitting files through an HTML form.

20. What is the purpose of `autocomplete`?

It allows the browser to offer or manage suggested values for form fields.

21. What is the purpose of the `form` attribute on an input?

It associates a control with a form by the form's ID, even if the control is outside the form element.

22. What is the purpose of `formaction`?

It overrides the form's submission URL for a particular submit control.

23. What is the purpose of `formmethod`?

It overrides the form's submission method for a particular submit control.

24. What is the purpose of `formnovalidate`?

It allows a particular submit control to bypass the browser's built-in constraint validation.

25. Does HTML validation replace backend validation?

No. Client-side validation improves usability, but the server must validate incoming data because client-side checks can be bypassed.

Final Revision Checklist

- [ ] Understand the purpose of HTML forms.
- [ ] Use `action` and `method` correctly.
- [ ] Explain GET vs. POST.
- [ ] Understand form encoding and file uploads.
- [ ] Connect labels to controls.
- [ ] Use text, email, password, number, date, and file inputs.
- [ ] Understand radio buttons and checkboxes.
- [ ] Create dropdowns and textareas.
- [ ] Use fieldsets and legends.
- [ ] Apply required, minlength, maxlength, min, max, step, and pattern.
- [ ] Understand readonly vs. disabled.
- [ ] Understand form, formaction, formenctype, formmethod, formnovalidate, and formtarget.
- [ ] Build a complete student registration form.

- [ ] Understand that real submission requires a suitable backend.
- [ ] Validate submitted data on the server.

Recommended learning order

1. Start with simple forms and labels.
2. Practice input types.
3. Learn validation attributes.
4. Practice GET and POST.
5. Learn file uploads and form-specific submit attributes.
6. Build the student registration project.
7. Later, connect the form to a backend using a technology such as Java, Python Flask, PHP, or Node.js.